

III. Authentication and Authorization

1. Нови понятия: автентикация и оторизация

- **Автентикация:** Процесът на проверка на самоличността на потребителя (логин/регистрация).
- **Оторизация:** Определяне на нивото на достъп на потребителя до различни части на приложението.

Ще добавим следните **роли** за управление на достъпа:

- **Гост:** Може само да разглежда каталога с книги.
 - **Регистриран потребител:** Може да добавя и редактира книги.
 - **Администратор:** Може да добавя, редактира и изтрива книги.
-

2. Добавяне на Identity в проекта

2.2. Конфигуриране на Identity в Program.cs

Добавете следния код в Program.cs

```
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
options.SignIn.RequireConfirmedAccount = false)
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

```
21 builder.Services.AddDefaultIdentity<IdentityUser>(options =>
22 options.SignIn.RequireConfirmedAccount = false)
23     .AddRoles<IdentityRole>()
24     .AddEntityFrameworkStores<ApplicationDbContext>();
```

Допълнителни опции за username, password:

```
//конфигуриране на автентикация,пароли и роли
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = true; // Изисква поне една цифра
    options.Password.RequiredLength = 4; // Минимална дължина (може да се промени)
    options.Password.RequireLowercase = false; // Не изисква малки букви
    options.Password.RequireUppercase = false; // Не изисква главни букви
    options.Password.RequireNonAlphanumeric = false; // Не изисква специални символи
})
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
builder.Services.AddDefaultIdentity<IdentityUser>(options =>
```

```

{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = true; // Изисква поне една цифра
    options.Password.RequiredLength = 4; // Минимална дължина
    options.Password.RequireLowercase = false; // Не изисква малки букви
    options.Password.RequireUppercase = false; // Не изисква главни букви
    options.Password.RequireNonAlphanumeric = false; // Не изисква
специални символи
})
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<ApplicationDbContext>();

```

2.3. Изпълнение на миграции

Add-Migration AddIdentity
Update-Database

3. Добавяне на роли при стартиране на приложението

- Създайте метод за инициализиране на роли **CreateRoles** в **Program.cs** след **pp.Run()**:

```

async Task CreateRoles(IServiceProvider serviceProvider)
{
    var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
    var userManager = serviceProvider.GetRequiredService<UserManager<IdentityUser>>();

    string[] roleNames = { "Admin", "User" };
    foreach (var roleName in roleNames)
    {
        if (!await roleManager.RoleExistsAsync(roleName))
        {
            await roleManager.CreateAsync(new IdentityRole(roleName));
        }
    }
    // Създаване на администраторски акаунт
    string adminEmail = "admin@gmail.com";
    string adminPassword = "admin123";

    var adminUser = await userManager.FindByEmailAsync(adminEmail);
    if (adminUser == null)
    {
        adminUser = new IdentityUser { UserName = adminEmail, Email = adminEmail };
        var result = await userManager.CreateAsync(adminUser, adminPassword);
        if (result.Succeeded)
        {
            await userManager.AddToRoleAsync(adminUser, "Admin");
        }
    }
}

```

- Извикайте този метод в `Program.cs` преди `app.Run()`:

```
using (var scope = app.Services.CreateScope())
{
    var roleManager =
scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    if (!await roleManager.RoleExistsAsync("User"))
    {
        await roleManager.CreateAsync(new IdentityRole("User"));
    }
    var service = scope.ServiceProvider;
    await CreateRoles(service);
}
```

- Общ вид на `Program.cs`:
Обърнете внимание че самия метод `Main()` е модифициран добавени са **async Task**, за да може да работи асинхронният метод `CreateRoles()` !!!

0 references

```
public static async Task Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);

    // *****
    var connectionString = builder.Configuration.GetConnectionString("DefaultConn

    builder.Services.AddDbContext<ApplicationDbContext>(options =>
        options.UseSqlServer(connectionString));

    builder.Services.AddDatabaseDeveloperPageExceptionFilter();

    //*****
    builder.Services.AddDefaultIdentity<IdentityUser>(options =>
    {
        options.SignIn.RequireConfirmedAccount = false;

builder.Services.AddDefaultIdentity<IdentityUser>(options =>
{
    options.SignIn.RequireConfirmedAccount = false;
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
    options.Password.RequireLowercase = false;
    options.Password.RequireUppercase = false;
    options.Password.RequireNonAlphanumeric = false;
}))

    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>();
```

```
app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapRazorPages();
```

```
using (var scope = app.Services.CreateScope())
{
    var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();

    if (!await roleManager.RoleExistsAsync("User"))
    {
        await roleManager.CreateAsync(new IdentityRole("User"));
    }
    var service = scope.ServiceProvider;
    await CreateRoles(service);
}
```

```
app.Run();
```

```
async Task CreateRoles(IServiceProvider serviceProvider)
{
    var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
```

4. Ограничаване на достъпа до контролера

4.1. Актуализиране на BooksController

```

[Authorize]
1 reference
public class BooksController : Controller
{
    private readonly ApplicationDbContext _context;

    0 references
    public BooksController(ApplicationDbContext context)
    {
        _context = context;
    }
    // GET: Books
    [AllowAnonymous]//Разрешава на гостите да разглеждат каталога
    3 references
    public async Task<IActionResult> Index()
    {
        return View(await _context.Books.ToListAsync());
    }
    [Authorize(Roles = "User,Admin")]
    // GET: Books/Details/5
    0 references
    public async Task<IActionResult> Details(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var book = await _context.Books
            .FirstOrDefaultAsync(m => m.Id == id);
    }
}

```

5. Добавяне на навигация за вход/изход

Във `Views/Shared/_Layout.cshtml`, добавете:

```

@if (User.Identity.IsAuthenticated)
{
    <li class="nav-item">
        <a class="nav-link text-dark">Здравей, @User.Identity.Name!</a>
    </li>
    <li class="nav-item">
        <form method="post" asp-area="Identity" asp-page="/Account/Logout">
            <button type="submit" class="btn btn-link nav-link text-dark">Изход</button>
        </form>
    </li>
}
else
{
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Login">Вход</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Register">Регистрация</a>
    </li>
}

```

```

<ul class="navbar-nav flex-grow-1">
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
  </li>

  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Books" asp-action="Index">Books</a>
  </li>
  @if (User.Identity.IsAuthenticated)
  {
    <li class="nav-item">
      <a class="nav-link text-dark">Здравей, @User.Identity.Name!</a>
    </li>
    <li class="nav-item">
      <form method="post" asp-area="Identity" asp-page="/Account/Logout">
        <button type="submit" class="btn btn-link nav-link text-dark">Изход</button>
      </form>
    </li>
  }
  else
  {
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Login">Вход</a>
    </li>
    <li class="nav-item">
      <a class="nav-link text-dark" asp-area="Identity" asp-page="/Account/Register">Регистрация</a>
    </li>
  }
</ul>

```

6. Тестване на приложението

1. Стартирайте проекта (F5).
 2. Опитайте да разглеждате книгите без вход – трябва да работи.
 3. Регистрирайте нов потребител и опитайте да добавите книга – трябва да работи.
 4. Влезте с admin@gmail.com / **admin123** и опитайте да изтриете книга – трябва да работи.
-